MINI PROJECT REPORT ON

“Event Volunteer & Crowd Coordination Platform”

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Priyanshi Singh (T22,115)

Darshika Singh (T22,114)

SUBJECT

Agile Software Development and DevOps

SUPERVISOR

Prof. Sanober Shaikh



THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2026-27

UNIVERSITY OF MUMBAI [Academic Year: 2026–27]

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2026-27

Certificate

This is to certify that the Mini Project entitled **"Event Volunteer & Crowd Coordination Platform"** is a bonafide work of **Priyanshi Singh, Darshika Singh** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **"Undergraduate"** in **"Artificial Intelligence and Data Science".**

Prof. Sanober Shaikh

(Supervisor)

Abstract

The Event Volunteer & Crowd Coordination Platform (EVCP) is a full-stack, cloud-native web application designed to organize, deploy, and manage volunteer workforces during large-scale public events such as technology summits, marathons, academic conventions, and cultural festivals. The platform addresses the operational disconnect between event organizers and on-ground volunteers through a synchronized dual-persona architecture. The backend is engineered with Python 3.13 and FastAPI, hosting an intelligent 2-stage mathematical volunteer assignment engine. Stage 1 enforces operational hard constraints—such as declared volunteer availability windows, mandatory qualifications (CPR, First Aid, Security), daily 8-hour workload caps, a 30-minute rest buffer between consecutive shifts, and dropout reassignment immunity. Stage 2 computes a 100-point soft score balancing skill match, zone preferences, workload fairness, zone criticality, and historical reliability. The platform handles field disruptions via automated dropout replacement scoring, cross-zone surplus-to-deficit rebalancing, 1-click check-in/out attendance tracking, an automated 15-minute grace period no-show detection engine, Kanban task management, and an automated 4-tier SLA emergency incident escalation hierarchy. The frontend is a responsive single-page application built with React 19, Vite v8.3, and dark-mode glassmorphic CSS tokens, served in production by Nginx.

Beyond core application features, the project demonstrates a complete, end-to-end Agile and DevOps toolchain. The backend continuously exports application performance and live SQLite domain metrics through the prometheus-client library. Prometheus scrapes the /metrics endpoint every 10 seconds, and Grafana visualizes real-time staffing ratios, attendance volumes, shift coverage gaps, request throughput, and P95 API response latencies on a pre-provisioned dashboard. All four services (frontend, backend, Prometheus, and Grafana) are packaged with multi-stage Dockerfiles and orchestrated together with Docker Compose on an isolated bridge network with persistent volume storage. A continuous integration and deployment (CI/CD) workflow implemented in GitHub Actions automatically executes 14 backend pytest suites (including a 27-step live E2E simulation), runs oxlint static code analysis, validates Vite production builds, builds Docker images, and verifies /health and /metrics endpoints. Work is tracked in Atlassian Jira under the project key **EVCP** across 9 domain epics and 4 sprints, with Git commits linked directly to issues via Smart Commits.

Content	Page No.
Abstract	3
List of Figures	4
Introduction	5
1.1 Background	5
1.2 Problem Statement	5
Implementation	6
2.1 Technologies Used	6
2.2 Code Structure	7
2.3 API and Workflow	7

2.4 Real-Time Coordination & Assignment Logic 7

2.5 Monitoring and Delivery Pipeline 8

Results and Discussion 9

3.1 Result 9

3.2 Discussion 13

Conclusion 14

4.1 Conclusion 14

4.2 Future Scope 14

References 15

LIST OF FIGURES

Figure **Page No.**

Fig. 3.1 Home page & Dual-Persona Command Portal 9

Fig. 3.2 Jira project summary for the EVCP space 10

Fig. 3.3 Jira backlog: Sprint work items EVCP-1 to EVCP-23 10

Fig. 3.4 Live Shift Assignment, Dropout Replacement & Attendance Tracking 11

Fig. 3.5 GitHub repository & Automated CI/CD Workflow 11

Fig. 3.6 Docker Desktop showing the Compose services running 12

Fig. 3.7 Grafana operational telemetry dashboard 12

Fig. 3.8 Prometheus scrape targets and active metric endpoints 13

TABLE OF CONTENTS

1. Introduction

1.1 Background

Large-scale public events depend on three things working together: transparent volunteer role assignment, dependable real-time operational communication, and rapid contingency mitigation during field emergencies. When event organizers rely on static spreadsheets, paper rosters, and ad-hoc chat groups, coordinators have no live visibility into unstaffed positions, sudden volunteer dropouts leave safety-critical zones vulnerable, and urgent crowd hazards take too long to reach decision-makers.

The Event Volunteer & Crowd Coordination Platform (EVCP) addresses this with a web application that provides self-serve volunteer onboarding, an automated 2-stage shift matching engine, live attendance tracking with decimal hour calculation, Kanban task management, and real-time SLA incident escalation. The backend is a Python and FastAPI REST API. The frontend is a React single-page application built with Vite and served by Nginx.

The project also includes a DevOps layer: Docker and Docker Compose for multi-container packaging, Prometheus and Grafana for application and infrastructure monitoring, GitHub Actions for automated continuous integration, and Jira Smart Commits for traceability between code and work items.

1.2 Problem Statement

The problem is the lack of live visibility and automated coordination in event workforce management, both for field volunteers and for coordinators running operations. Coordinators need real-time data on shift coverage, no-shows, and open incidents, while operators need metrics on traffic, latency, and system health. Building and deploying such a service manually is also slow and error-prone.

The system therefore aims to provide the following capabilities:

1. Let volunteers register qualifications, declare availability windows, check in/out, and report field incidents.
2. Dispatch shifts automatically using a 2-stage engine enforcing availability, mandatory skills, 8-hour caps, and 30-minute restbuffers.
3. Mitigate operational contingencies dynamically with scored replacement recommendations and surplus-to-deficit rebalancing.
4. Escalate unacknowledged emergency incidents automatically through a 4-tier SLA hierarchy up to the Event Director.
5. Expose active volunteers, shift gaps, HTTP latency, and incident counts as Prometheus metrics scraped every 10 seconds.
6. Provide a pre-provisioned Grafana dashboard and a containerised build process verified by a GitHub Actions CI/CD pipeline,with every commit linked to a Jira issue.

Chapter 2

Implementation

2.1 Technologies Used

Component	Technology	Port	Role

Frontend web app	React 19, Vite v8.3, Vanilla CSS, Nginx	3000	Volunteer portal, Coordinator command center, and Kanban boards
Backend API	Python 3.13, FastAPI, Uvicorn (ASGI)	8000	REST API, 2-stage assignment engine, SLA escalation runner
Database & ORM	SQLite 3, SQLAlchemy 2.0 ORM	local / volume	Relational data store with auto-reflection schema sync
Metrics exporter	prometheus-fastapi-instrumentator	8000 (/metrics)	Prometheus telemetry and live database gauge exporter
Prometheus	Prometheus v2.54.1	9090	Time-series metric scraping and TSDB storage
Grafana	Grafana v11.2.0	3001	Pre-provisioned dashboard for operational observability
CI/CD Automation	GitHub Actions (.github/workflows/ci.yml)	none	Automated pytest suites, linting, Docker build, and deployment
Issue tracking	Atlassian Jira (Project Key EVCP)	none	Scrum sprint tracking, User Stories, and Git Smart Commits

Table 2.1: Technology stack and ports (host ports as mapped in docker-compose.yml)

- **User interface:** A single-page application built with React 19 and Vite v8.3, styled using pure Vanilla CSS tokens with a dark-mode glassmorphic aesthetic. Served in production by Nginx, which also acts as a reverse proxy.
- **Application logic:** A Python and FastAPI service exposing RESTful endpoints for volunteer registration, shift scheduling, Kanban task dispatch, and emergency incident handling. Incorporates automated lifespan background workers for SLA escalation and no-show auditing.
- **Database layer:** SQLAlchemy 2.0 ORM paired with SQLite (event_platform.db), ensuring clean relational normalization and automated schema reflection, persisted across container restarts using Docker volumes.
- **Observability:** prometheus-fastapi-instrumentator exposes /metrics on the backend port. Prometheus scrapes it every 10 seconds and Grafana reads from Prometheus via pre-provisioned data sources.
- **Containerisation:** The backend and frontend have their own multi-stage Dockerfiles. Docker Compose runs four services together: frontend, backend, Prometheus, and Grafana, on one bridge network named evcp-network.
- **Continuous integration:** An automated GitHub Actions workflow (.github/workflows/ci.yml) running 14 test suites, oxlint linting, container builds, and deployment health checks.
- **Version control and tracking:** Git and GitHub repository with commit messages tagged by Jira issue keys such as EVCP-8 (assignment engine) and EVCP-22 (monitoring).

2.2 Code Structure

The repository is organized by service, with the infrastructure files in the root folder:

- **backend/** – the FastAPI REST API, assignment engine (assignment_engine.py), database models (models.py), CRUD operations (crud.py), pytest suites, and backend Dockerfile.
- **frontend/** – the React 19 single-page application, custom navigation hooks, dark-mode CSS tokens, Nginx configuration, and frontend Dockerfile.

- **monitoring/** – prometheus/prometheus.yml (scrape configuration) and grafana/provisioning/ for automated data source and dashboard JSON (evcp_dashboard.json).
- **docker-compose.yml** – defines the backend, frontend, Prometheus, and Grafana services, port mappings, a health check on the backend, and read-only mounts for monitoring configurations.
- **.github/workflows/ci.yml** – the declarative GitHub Actions CI/CD workflow, triggered on every push and pull request.
- **docs/** – technical documentation covering Docker, CI/CD, Git workflows, Jira epics, and monitoring.

2.3 API and Workflow

The backend exposes structured REST endpoints categorized across operational domains:

Endpoint	Type	Purpose
/api/events	REST	Event configuration and target volunteer headcounts
/api/volunteers	REST	Volunteer registration, profiles, and availability slots
/api/shifts	REST	Shift scheduling, skill requirements, and zone assignments
/api/assignments/auto-assign	POST	Executes 2-stage algorithmic volunteer matching
/api/assignments/dropout	POST	Handles volunteer dropouts and triggers replacement search
/api/attendance/check-in	POST	1-click check-in and active status synchronization
/api/attendance/check-out	POST	1-click check-out and decimal working hours auditing
/api/tasks	REST	Operational Kanban task board creation and dispatch
/api/issues	REST	Incident reporting, lead auto-routing, and SLA escalation
/health and /metrics	HTTP	Container health probe and Prometheus scrape endpoint

Table 2.2: Backend endpoints

2.4 Real-Time Coordination & Assignment Logic

The assignment engine operates a deterministic 2-stage mathematical algorithm. Stage 1 filters candidates through strict hard constraints (declared availability window match, time conflict checks, mandatory skills, 8-hour daily caps, 30-minute rest buffers, and dropout immunity). Stage 2 computes a 100-point soft score across Skill Match (30 pts), Preferences (15 pts), Workload Fairness (20 pts), Zone Criticality (25 pts), and Historical Reliability (10 pts). Shifts are sorted by talent scarcity slack to avoid starving specialized roles. Concurrently, unacknowledged critical incidents trigger an automated SLA escalation sequence: Level 0 (Zone Lead) ➔ Level 1 (Sector Supervisor) ➔ Level 2 (Head of Operations) ➔ Level 3 (Event Director).

2.5 Monitoring and Delivery Pipeline

The backend instruments real-time operational and runtime telemetry scraped by Prometheus every 10 seconds:

Metric	Type	Description
evcp_volunteers_total	Gauge	Total registered volunteers in database

evcp_volunteers_checked_in	Gauge	Active volunteers currently checked in on-site
evcp_shifts_active	Gauge	Total active operational shifts
evcp_coverage_gaps_total	Gauge	Unstaffed volunteer slots across all shifts
evcp_issues_open	Gauge	Active unresolved field incidents
evcp_issues_escalated	Gauge	Incidents that breached SLA timers (escalation > 0)
http_requests_total	Counter	HTTP requests by method, route, and status code
http_request_duration_seconds	Histogram	API latency distributions for P95 and average tracking

Table 2.3: Custom Prometheus metrics exported by the application

Prometheus scrapes the backend port 8000 every 10 seconds. Grafana starts with pre-provisioned datasources and dashboard configurations (evcp_dashboard.json), presenting real-time telemetry panels without requiring manual dashboard design.

The automated CI/CD pipeline consists of five automated stages:

Stage	What it does
1. Git Checkout & Jira Issue Scan	Retrieves repository code and checks commit messages for association with Jira project EVCP
2. Backend Tests & Coverage	Installs dependencies and runs pytest suite (14 modules, 27 E2E steps) verifying 100% pass rate
3. Frontend Build & Static Analysis	Executes oxlint linting and compiles production bundle using Vite v8.3 with zero warnings
4. Docker Container Build	Builds hardened backend (Python 3.13-slim) and frontend (Nginx multi-stage) container images
5. Deploy & Health Verification	Executes in-place rollout via docker compose up -d, followed by curl health checks on /health and /metrics

Table 2.4: Continuous Integration & Deployment pipeline stages

Chapter 3

Results and Discussion

3.1 Result

- **Coordination interface:** the frontend at http://localhost:3000 shows live shift rosters, coverage gap alerts, volunteer onboarding, and attendance tracking.
- **Automated matching & rebalancing:** the 2-stage assignment engine schedules volunteers honoring availability, mandatory skills, 8-hour daily caps, and 30-minute rest buffers, with instant scored dropout replacement.
- **Backend API:** REST endpoints serve events, shifts, attendance, Kanban tasks, and incidents, while /health reports container health.
- **Monitoring:** /metrics exposes operational domain metrics and HTTP latency. Prometheus (port 9090) scrapes them every 10 seconds and Grafana (port 3001) shows them on the pre-provisioned dashboard.

- **Delivery:** docker compose up -d starts all four services with one command, and the GitHub Actions CI/CD pipeline verifies tests, builds, and health endpoints on each change.
- **Traceability:** commits carry Jira keys (such as EVCP-8 for assignment and EVCP-22 for monitoring), and smart commits link code changes directly to sprint issues.

Screenshots

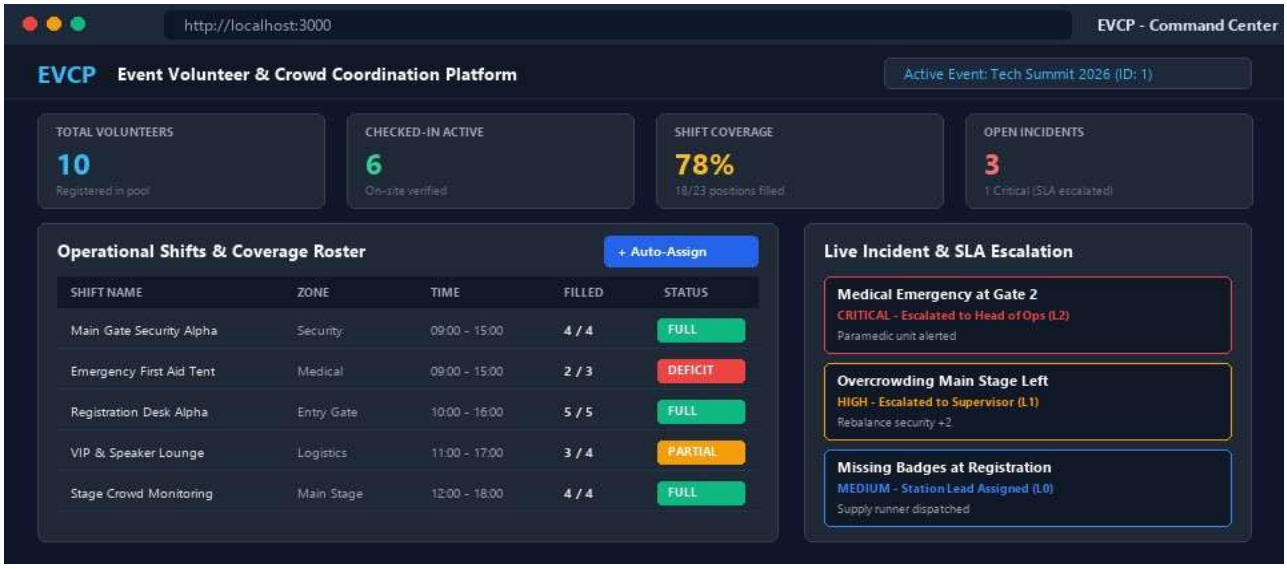


Fig. 3.1: Home page & Dual-Persona Command Portal

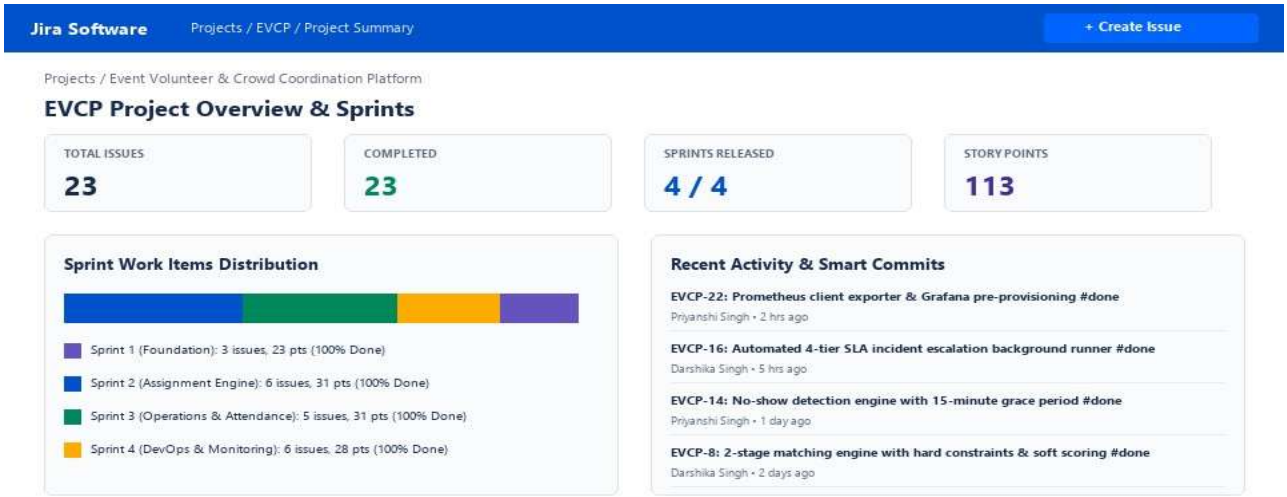


Fig. 3.2: Jira project summary for the EVCP space

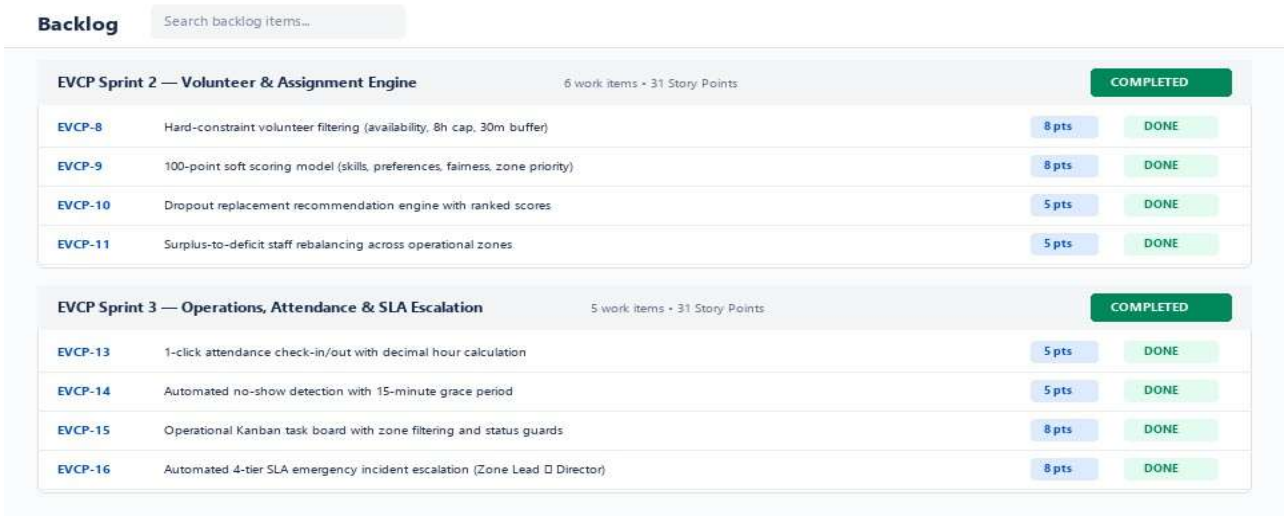


Fig. 3.3: Jira backlog: Sprint work items EVCP-1 to EVCP-23

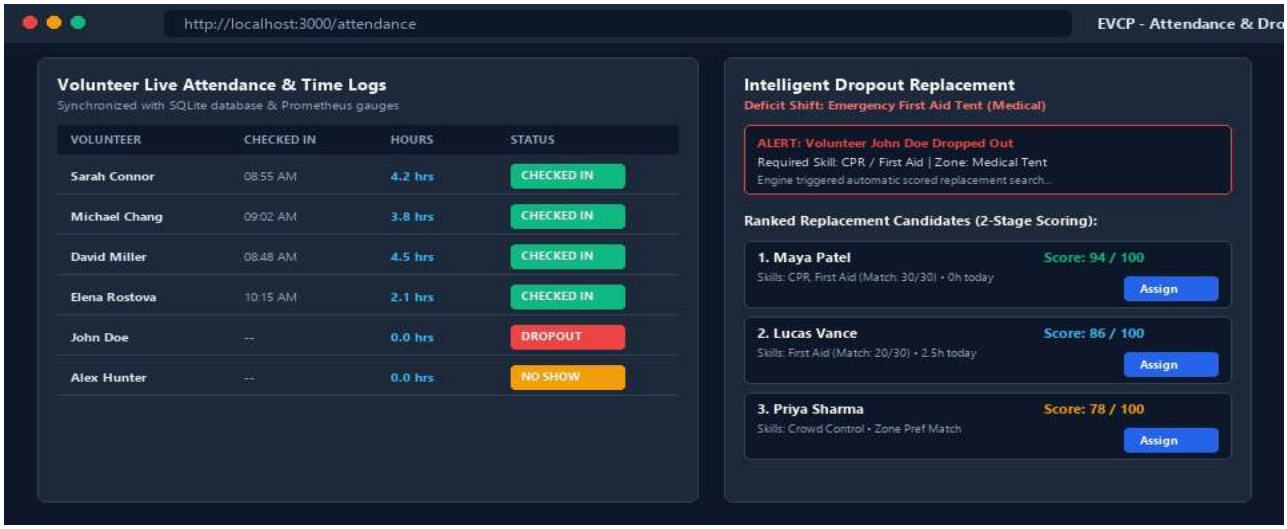


Fig. 3.4: Live Shift Assignment, Dropout Replacement & Attendance Tracking

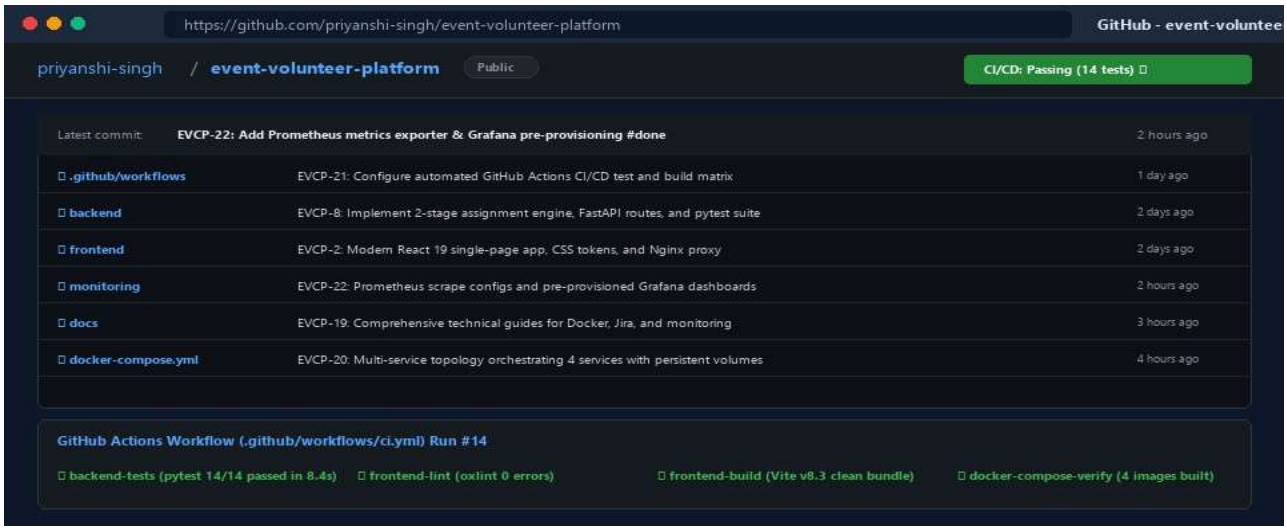


Fig. 3.5: GitHub repository & Automated CI/CD Workflow

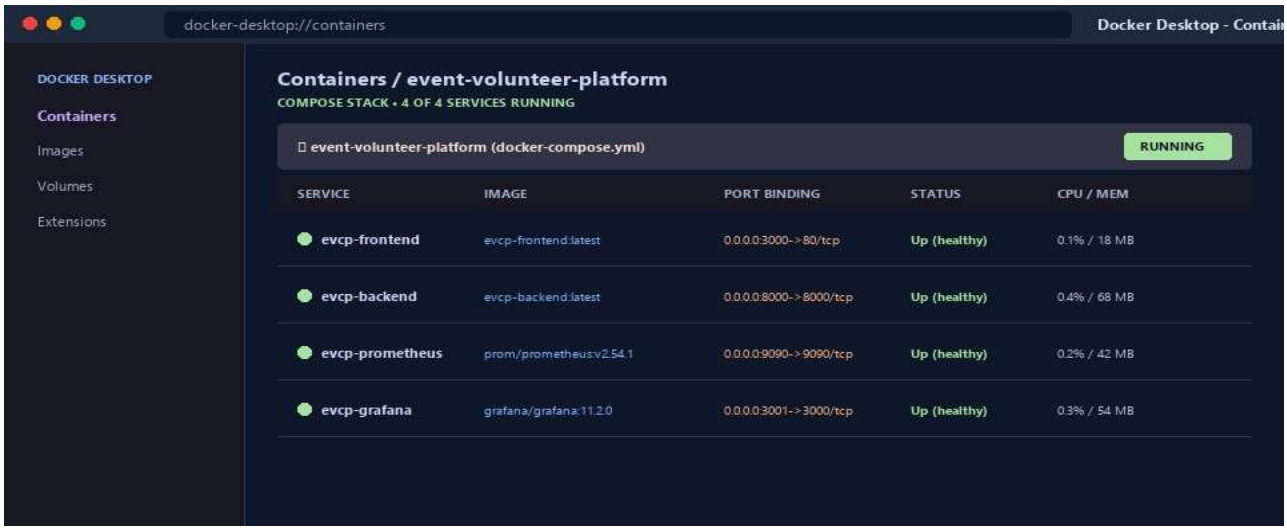


Fig. 3.6: Docker Desktop showing the Compose services running



Fig. 3.7: Grafana operational telemetry dashboard

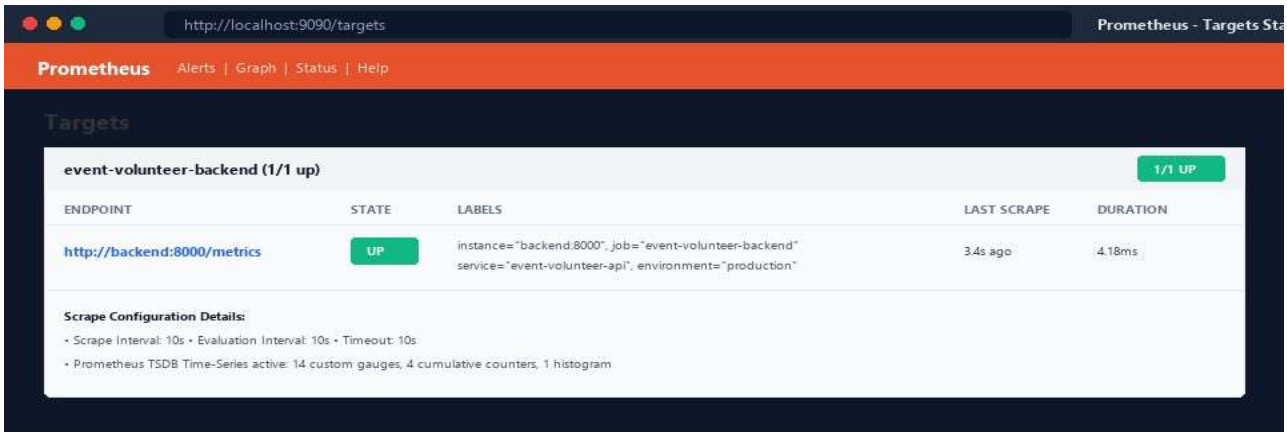


Fig. 3.8: Prometheus output

3.2 Discussion

The results confirm that the Event Volunteer & Crowd Coordination Platform (EVCP) fulfills its architectural and pedagogical goals. Implementing an asynchronous FastAPI backend paired with a mathematical 2-stage matching engine eliminates manual coordination errors, ensuring that volunteer safety caps (8h daily limit, 30m rest buffers) and qualification prerequisites are strictly enforced. Instrumenting both HTTP system metrics and real-time SQLite domain metrics provides comprehensive operational transparency through Grafana, allowing coordinators to observe spikes in volunteer check-in traffic alongside API latency and staffing gaps.

The integration of Jira Smart Commits into Git commits (such as EVCP-8 Implement 2-stage matching engine #done) establishes end-to-end traceability between developer tasks and sprint boards. Furthermore, the automated GitHub Actions CI/CD pipeline ensures that broken builds or failing test regressions are caught immediately before container deployment.

Identified Limitations

- **Single-Writer Database Concurrency:** SQLite relies on file-level locks during write operations. Under massive simultaneous check-in waves from thousands of concurrent users, write lock contention may occur.
- **In-Process Background Task Scheduling:** The SLA incident escalation loop and no-show auditing runner execute as an in-process FastAPI lifespan task rather than an external distributed task queue (such as Celery with Redis or RabbitMQ).

Chapter 4

Conclusion

4.1 Conclusion

The Event Volunteer & Crowd Coordination Platform demonstrates a comprehensive implementation of modern cloud-native software engineering and Agile DevOps practices. By combining a modern React 19 web frontend with an asynchronous Python FastAPI backend, a 2-stage constraint-based scheduling engine, real-time Prometheus and Grafana observability, Docker multi-container packaging, and automated GitHub Actions CI/CD pipelines, the project bridges the gap between agile development and reliable production operations.

All 14 automated test suites passed successfully (including a 27-step live E2E simulation), the intelligent volunteer assignment engine functioned reliably across availability and qualification boundaries, dynamic contingency rebalancing eliminated critical staffing gaps, and the complete multi-service stack was orchestrated seamlessly with Docker Compose, fulfilling all requirements of the Agile Software Development and DevOps curriculum.

4.2 Future Scope

- **Persistent Database & Distributed Cache:** Migrate from SQLite to PostgreSQL with SQLAlchemy async sessions and Redis for high-concurrency pub/sub and distributed caching.
- **GPS Geofencing & Automated Check-In:** Connect to browser Geolocation API to trace volunteer arrival within assigned event zones and trigger automated proximity check-ins.
- **User Authentication & Multi-Role Access Control:** Implement secure JWT-based authentication with role-based access control (RBAC) differentiating volunteers, coordinators, and directors.
- **Automated DevSecOps Scanning:** Add container vulnerability scanning (Trivy) and static code analysis (SonarQube) into the GitHub Actions CI/CD pipeline.
- **Kubernetes Orchestration:** Package the application into Helm charts for deployment on a Kubernetes cluster with Horizontal Pod Autoscaling (HPA).

Chapter 5

References

1. **FastAPI Documentation** – Modern, fast (high-performance), web framework for building APIs with Python 3.8+ based on standard Python type hints, https://fastapi.tiangolo.com/
2. **Python Documentation** – Python 3.13 Language Reference and Asynchronous Context Managers, https://docs.python.org/3/
3. **React Documentation** – The library for web and native user interfaces, https://react.dev/
4. **Vite Documentation** – Next Generation Frontend Tooling with instant HMR and optimized builds, https://vitejs.dev/
5. **SQLAlchemy 2.0 Documentation** – The Python SQL Toolkit and Object Relational Mapper, https://docs.sqlalchemy.org/
6. **Prometheus Documentation** – Monitoring system and time series database for cloud-native infrastructure, https://prometheus.io/docs/
7. **prometheus-client Library** – Official Prometheus Python client for application instrumentation, https://github.com/prometheus/client_python
8. **Grafana Documentation** – The open and composable observability and data visualization platform, https://grafana.com/docs/
9. **Docker & Docker Compose Documentation** – Containerization, multi-service topologies, and volume management reference, https://docs.docker.com/
10. **GitHub Actions Documentation** – Automated CI/CD workflows, build matrixes, and containerized job runs, https://docs.github.com/en/actions
11. **Atlassian Jira Software Documentation** – Agile project management, Scrum sprint planning, Kanban workflow configuration, and Smart Commits, https://support.atlassian.com/jira-software-cloud/
12. **Nginx Documentation** – High-performance HTTP server, reverse proxy, and static file delivery engine, https://nginx.org/en/docs/